

WebFlux Demo (JDK 11) — WebFlux + Reactive MongoDB

Documento técnico de referencia del proyecto incluido en el ZIP: arquitectura, flujo request/response, errores tipados, validación, OpenAPI y observabilidad.

1. Qué es WebFlux y cómo se usa acá

Spring WebFlux es el stack reactivo de Spring basado en Reactor (Mono/Flux) y un runtime non-blocking (Reactor Netty por defecto). El proyecto expone APIs de ejemplo por dos estilos: (a) controllers anotados y (b) functional routes (RouterFunction/Handler).

En ambos casos, el contrato es reactivo: los handlers devuelven Mono/Flux y no bloquean el event-loop. La persistencia se hace con MongoDB en modo reactivo (ReactiveMongoRepository).

2. Estructura del proyecto

```
src/main/java/com/example/webflux
WebfluxDemoApplication.java
config/
  WebFluxConfig.java           (CORS + habilita MDC en Reactor)
  DataInitializer.java         (datos demo al arrancar)
controller/
  ProductoController.java      (REST v1 con @RestController)
router/
  ProductoRouter.java          (REST v2 functional routes)
handler/
  ProductoHandler.java         (handlers funcionales)
  GlobalErrorWebExceptionHandler.java (error handler global JSON)
model/
  Producto.java                (@Document Mongo)
dto/
  ProductoRequest.java
  ProductoResponse.java
repository/
  ProductoRepository.java      (ReactiveMongoRepository)
service/
  ProductoService.java
  WebClientService.java
validation/
  ValidationSupport.java       (validación consistente para functional)
observability/
  CorrelationIdWebFilter.java  (X-Correlation-Id + Reactor Context)
  ReactorMdc.java              (propagación MDC)
```

3. Persistencia reactiva con MongoDB

Se usa **spring-boot-starter-data-mongodb-reactive**. El repositorio es ReactiveMongoRepository y retorna Mono/Flux. Esto evita JDBC/R2DBC en este ejemplo y simplifica la infraestructura para JDK 11.

Configuración: **spring.data.mongodb.uri** por perfil (dev/test/docker/prod) en application.yml.

4. Excepciones tipadas + handler global

La capa service lanza excepciones tipadas: NotFoundException, BadRequestException y ConflictException. El handler global **GlobalErrorWebExceptionHandler** mapea la excepción a un status

HTTP y retorna un JSON estable (timestamp, status, error, message, path).

5. Validación consistente en functional routes

En controllers anotados se usa `@Valid`. En functional routes no hay `@Valid` automático sobre el body, por eso se incluyó **ValidationSupport** que usa `javax.validation.Validator` para validar el DTO y lanza `BadRequestException` con un mensaje consolidado.

6. OpenAPI / Swagger (springdoc webflux)

Se incluye **springdoc-openapi-webflux-ui**. URLs:

```
Swagger UI:    http://localhost:8080/swagger-ui.html
OpenAPI JSON:  http://localhost:8080/v3/api-docs
```

7. Observabilidad

Correlation-ID: `WebFilter` lee/crea `X-Correlation-Id`, lo agrega a response y lo inyecta en Reactor Context. Para logs, `ReactorMdc` propaga el valor al MDC para que aparezca en el JSON log.

Logs JSON: se usa `logstash-logback-encoder` en `logback-spring.xml`. Métricas: Actuator + Prometheus en `/actuator/prometheus` y tags custom (`application/env`) configurados en `management.metrics.tags`.

8. Tests (MongoDB con Testcontainers)

Los tests levantan un MongoDB real vía `Testcontainers`, y sobrescriben `spring.data.mongodb.uri` usando `DynamicPropertySource`. Incluye:

- `ProductoRepositoryTest (@DataMongoTest)`: prueba de persistencia reactiva.
- `ProductoControllerIT (@SpringBootTest + WebTestClient)`: smoke de endpoints y validación.

9. Ejecución rápida

```
# Mongo local
docker compose up -d mongo

# App (perfil dev)
mvn -q clean spring-boot:run -Dspring-boot.run.profiles=dev

# Tests
mvn -q test
```